

Banking Operating System — Development Plan

Product definition, architecture, security model and phased delivery roadmap

Prepared for Talenttalk IT

30 August 2026 · Version 1.0

Table of Contents

1. How to read this document

This is a development plan, not a sales brochure. It is written so that a developer can start building from it and a non-technical reader can follow what is being built and why.

Three things it deliberately does **not** do:

It does not promise a Temenos replacement. Temenos Transact, Oracle FLEXCUBE and Infosys Finacle each represent decades of engineering and thousands of person-years. Any plan that implies parity with them inside a normal project budget is misleading. What is achievable — and commercially interesting — is described in section 3.

It does not quote regulatory thresholds. Where a rule is named (GLBA, PCI DSS, PSD2 and so on) it is named because the control it demands shapes the architecture. Specific numeric thresholds, filing deadlines and reporting formats change and vary by jurisdiction; they must be confirmed with a compliance advisor licensed in the target market before the relevant module is built. Section 6 marks every place this applies.

It does not assume the UI video. The brief refers to a 23.9-second banking UI video as the visual direction. That file has not reached me — the message thread contains text only. The interface direction in this plan and in the working prototype is therefore my own, built to a stated design rationale (section 9) so it can be redirected cheaply once the video arrives. Nothing in the architecture depends on it.

Alongside this document there is a **working prototype** — a real application you can click through, not screenshots. It is described in section 10.

2. Is it worth building?

The brief asked me to assess this and then to proceed regardless. Here is the honest assessment, kept short.

2.1 Where the plan as originally framed fails

The blueprint circulated earlier lists roughly thirty modules: core banking, lending, payments hub, cards, treasury and ALM, trade finance, financial crime, four categories of risk, regulatory reporting, branch operations, digital banking, enterprise ERP, BI, AI copilots, a no-code product factory, migration tooling and country-specific regulatory architecture.

That list is a description of the incumbent vendors' *current* product surface — the result of twenty-plus years of accretion, much of it added because one large client demanded it. Building toward that list as a specification has a specific, well-documented failure mode: eighteen months of work produces thirty modules that are each 30% complete, and none of them is good enough to sell. The competitor you are copying, meanwhile, has moved.

There is a second problem. Core banking is not bought on features. It is bought on evidence — regulatory approval, audited controls, reference customers who have run a year-end close on the system, and a vendor the buyer believes will still exist in ten years. A feature-complete platform with none of that evidence does not win deals against Finacle. Its feature list is not the obstacle.

2.2 Where there is a real opportunity

The gap the incumbents leave is not depth. It is the **seam between systems**.

In most mid-size banks and non-bank financial institutions, the core holds the accounts, a separate system runs payments, finance closes the books in a spreadsheet, compliance works from CSV extracts, and the operations team spends its day reconciling the differences. That seam is where the cost, the errors and the audit findings live. It is also the part no incumbent has an incentive to fix, because each of them sells only one side of it.

A product that treats *the ledger, the operational workflow and the financial close as one system* has a defensible answer to a question the incumbents answer badly: **can you show me, right now, the evidence behind this number?**

That is the wedge, and it maps directly onto what the brief asks for — “make employee life easy, manager reporting easy, financial book closure”. Those are not secondary features. They are the product.

2.3 Where to start

Realistic first buyers are not Yes Bank, HDFC, Kotak or Axis. Those are three-to-five-year enterprise sales cycles requiring a track record you do not have yet. The realistic first buyers are:

Buyer type	Why they will look
Co-operative and small finance banks	Running genuinely old cores; small enough to decide
NBFCs and lending companies	Need a real ledger, currently on accounting software that is not one
Fintechs and neobanks	Need a core to build on; hostile to incumbent pricing and timelines
Microfinance institutions	Large transaction counts, thin margins, weak back-office tooling
Payment and wallet operators	Need double-entry settlement and reconciliation, not a full core

The strategy this plan follows: **build one vertical slice deep enough to run a real institution's back office, prove it in production with one customer, then widen.** Depth in one place beats breadth everywhere.

2.4 Verdict

Worth building — as a ledger-and-operations platform with a genuine financial close, sold initially to institutions the incumbents ignore. Not worth building as a Finacle clone. The rest of this plan assumes the former.

3. Product definition

3.1 One sentence

A banking operating system in which every customer, account, transaction, approval and closing entry lives on one immutable double-entry ledger, so that operations, finance and compliance work from the same record instead of reconciling three copies of it.

3.2 The four properties that must hold

Everything else is negotiable. These are not.

1. The ledger is immutable. Entries are appended, never updated or deleted. A correction is a reversal that posts its own entry and carries a reference to what it reverses. Every entry stores the hash of the entry before it, so the journal is a chain — altering history breaks the chain visibly and provably.

2. Nothing posts out of balance. Debits must equal credits at the moment of posting, enforced inside the database transaction. Not at end of day. Not at close. An unbalanced entry is impossible to persist, so “the ledger is out by ₹4,200” is not a state the system can reach.

3. Segregation of duties is enforced by the platform. The user who creates a payment can never release it, regardless of role, seniority or transaction limit. This is enforced in the service layer, not the interface, so it holds when the API is called directly.

4. Evidence attaches to the record. Supporting documents are attached to the transaction, the customer or the close task itself — not filed in a shared drive. The audit file assembles as work happens, rather than being reconstructed in the week before the auditor arrives.

3.3 Module map and phase assignment

#	Module	Phase	Notes
1	Identity, authentication, MFA	1	Password + TOTP, device binding, session policy
2	Roles, permissions, maker-checker	1	Platform-level, not UI-level
3	Customer 360 and KYC	1	Individual and corporate; document vault
4	Chart of accounts and GL	1	Configurable, multi-level
5	Immutable double-entry ledger	1	The spine of everything
6	Deposit accounts (current, savings)	1	Product parameters, interest accrual
7	Transactions and statements	1	Posting engine, running balance
8	Operations dashboard	1	KPIs, exception queue
9	Audit trail	1	Append-only, hash-chained
10	Financial close workflow	1	Trial balance, checklist, evidence, sign-

#	Module	Phase	Notes
			off
11	Reporting and export	1	Trial balance, P&L, balance sheet, statements
12	Term deposits	2	Maturity, renewal, premature closure
13	Payments hub — domestic rails	2	Rail-agnostic core; adapters per rail
14	Lending — origination to closure	2	Schedules, disbursement, repayment, delinquency
15	Reconciliation engine	2	Nostro, card network, rail settlement
16	Branch and teller operations	2	Cash drawer, vault, denomination, day open/close
17	Customer-facing digital banking	2	Web and mobile; read + payments
18	AML, sanctions, transaction monitoring	3	Rules engine, alert queue, disposition, SAR
19	Regulatory reporting	3	Per jurisdiction — build against a live requirement
20	Cards and wallets	3	Requires PCI DSS scope decisions first
21	Treasury, liquidity, ALM	3	Only with a customer who needs it
22	Corporate banking and trade finance	3	Large; defer until a buyer funds it
23	Credit, market	3	Framework in 3,

#	Module	Phase	Notes
	and operational risk		depth per customer
24	Enterprise ERP — AP, AR, procurement, assets	3	Reuses the ledger; genuinely differentiating
25	BI, analytics, profitability	3	Read replica, not the transaction database
26	Product factory / no-code configuration	3	Hard. Do not attempt before phase 3
27	Public API and event bus	2-3	API from phase 2; events as consumers appear
28	Migration from legacy cores	3	Sell as a service before productising it
29	AI assistance	3	See 3.4

3.4 A specific caution on AI features

“AI banking copilots” is the easiest item on the list to demo and the most dangerous to ship. A language model must never sit in the path of a financial posting, an approval decision, a credit decision or a compliance disposition — it cannot be audited, it is not deterministic, and a regulator will ask you to explain a decision you cannot explain.

Where it earns its place, and where phase 3 should use it:

- Natural-language search over the ledger and customer base (“show me every reversal over ₹10 lakh posted by the Mumbai branch last month”) — the model writes a query, the query runs, and the *query* is shown to the user
- Drafting variance commentary for the close, for a human to edit and sign
- Summarising a customer file, with every claim linked to its source document
- Triaging and clustering exception queues by suspected cause

In each case the model proposes and a human disposes, and the audit trail records the human. That distinction is the whole of it.

4. Architecture

4.1 Stack

Layer	Choice	Why
Language / framework	PHP 8.3 + Laravel 11	Matches the project's listed skill set; large hiring pool in India; mature queue, migration and testing tooling
Database	PostgreSQL 16	Serialisable isolation, exclusion constraints, native numeric — MySQL's defaults are wrong for ledgers
Cache / queue	Redis 7	Sessions, rate limits, queued jobs
Front end	Server-rendered Blade + Alpine.js; Vue 3 for dense screens	Fast to build, low JS surface; back-office users need speed, not a SPA
Search	PostgreSQL full-text initially; OpenSearch when volume demands	Do not add a second datastore before it is needed
Files	S3-compatible object storage, server-side encrypted	Documents never touch the application disk
Auth	Laravel + TOTP (RFC 6238); WebAuthn in phase 2	No SMS dependency, no per-message cost
Deployment	Docker; a single VM to start, Kubernetes only when scale justifies it	Premature orchestration is a common and expensive mistake

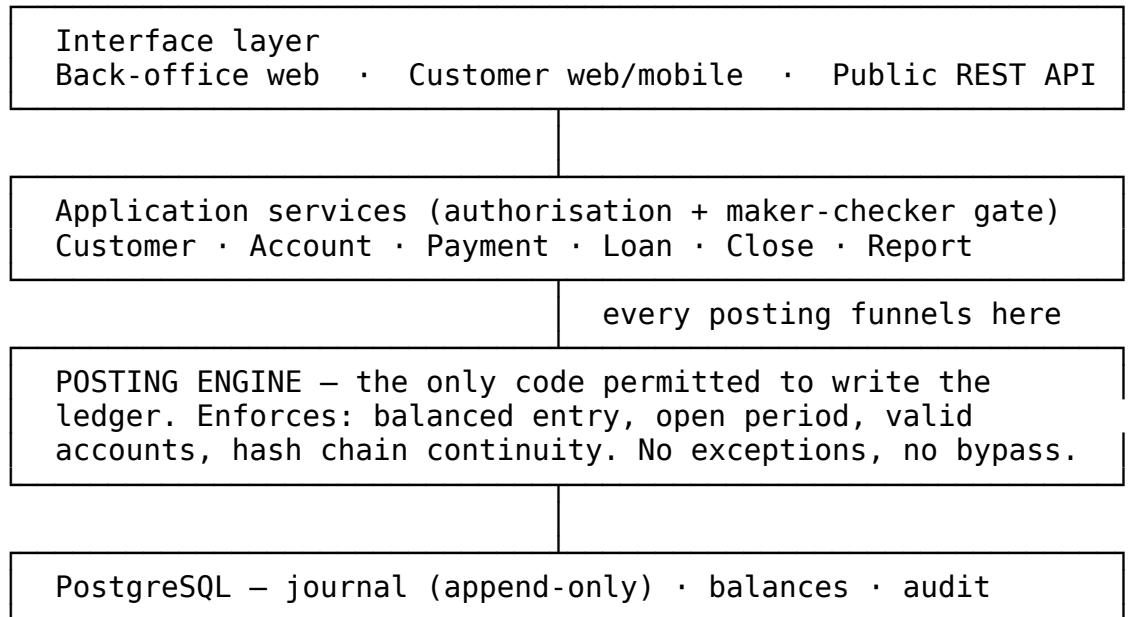
An alternative Node/NestJS + TypeScript stack is equally viable and no better. The database choice matters far more than the language choice.

4.2 A note on microservices

Do not start with microservices. Start with a well-partitioned monolith whose modules communicate through explicit service interfaces — the same boundaries a microservice split would use later, but with none of the distributed-transaction pain while the domain model is still moving.

The reason is specific to banking: a payment that debits an account, posts a journal entry, writes an audit record and enqueues a settlement instruction must be atomic. In one process that is a database transaction. Across four services it is a saga with compensating transactions, and every compensating transaction is a place a real bank loses real money. Split later, along boundaries proven by production load, and split the read-heavy parts first.

4.3 Module boundaries



The single most important structural rule: **one posting engine, no bypass**. Every module that moves money calls it. Nothing writes to the journal tables directly — enforced by database permissions, not by convention.

5. Data model

The core tables. Types are PostgreSQL.

5.1 Ledger

-- Chart of accounts. Hierarchical, configurable per institution.

```

CREATE TABLE gl_account (
  id          bigserial PRIMARY KEY,
  code        text NOT NULL UNIQUE,
  name        text NOT NULL,
  parent_id   bigint REFERENCES gl_account(id),
  type        text NOT NULL CHECK (type IN
    ('asset', 'liability', 'equity', 'income', 'expense')),
  normal_side char(1) NOT NULL CHECK (normal_side IN ('D', 'C')),

```

```

    is_postable    boolean NOT NULL DEFAULT true,
    currency       char(3),                -- NULL = multi-currency
    created_at     timestampz NOT NULL DEFAULT now()
);

-- Journal header. Append-only: no UPDATE, no DELETE (enforced by
-- grants).
CREATE TABLE journal_entry (
    id             bigserial PRIMARY KEY,
    reference      text NOT NULL UNIQUE,    -- JV-2026-08-30-00417
    business_date  date NOT NULL,
    posted_at      timestampz NOT NULL DEFAULT now(),
    narrative      text NOT NULL,
    source_module  text NOT NULL,          -- payments, lending,
    teller...
    posted_by      bigint NOT NULL REFERENCES app_user(id),
    reverses_id    bigint REFERENCES journal_entry(id),
    prev_hash      bytea NOT NULL,
    hash           bytea NOT NULL UNIQUE,
    CONSTRAINT no_self_reversal CHECK (reverses_id IS NULL OR
    reverses_id <> id)
);

-- Journal legs. Exactly one of debit/credit is non-zero.
CREATE TABLE journal_leg (
    id             bigserial PRIMARY KEY,
    entry_id       bigint NOT NULL REFERENCES journal_entry(id),
    gl_account_id  bigint NOT NULL REFERENCES gl_account(id),
    account_id     bigint REFERENCES account(id), -- customer account,
    if any
    currency       char(3) NOT NULL,
    debit          numeric(20,4) NOT NULL DEFAULT 0 CHECK (debit >= 0),
    credit         numeric(20,4) NOT NULL DEFAULT 0 CHECK (credit >= 0),
    fx_rate        numeric(18,8),
    base_debit     numeric(20,4) NOT NULL DEFAULT 0,
    base_credit    numeric(20,4) NOT NULL DEFAULT 0,
    CONSTRAINT one_side_only CHECK ((debit = 0) <> (credit = 0))
);
CREATE INDEX ON journal_leg (gl_account_id, entry_id);
CREATE INDEX ON journal_leg (account_id, entry_id);

```

Two decisions worth stating explicitly, because getting them wrong is expensive and getting them right is free:

numeric, never floating point. $0.1 + 0.2$ is not 0.3 in binary floating point. In a ledger that becomes a real, compounding discrepancy. Money is `numeric(20,4)` throughout, and no money value passes through a float or double at any point in the stack, including JSON serialisation.

Balances are derived, then cached — never authoritative. The journal is the truth. A balance table exists for performance, is updated inside the same transaction as the posting, and is reconciled against the journal on a schedule. If they ever disagree, the journal wins and the cache is rebuilt.

5.2 Customer and account

```
CREATE TABLE customer (  
  id          bigserial PRIMARY KEY,  
  cif         text NOT NULL UNIQUE,  
  type        text NOT NULL CHECK (type IN  
('individual', 'corporate')),  
  legal_name  text NOT NULL,  
  segment     text,  
  kyc_status  text NOT NULL DEFAULT 'pending'  
              CHECK (kyc_status IN  
('pending', 'verified', 'review_due', 'rejected')),  
  kyc_expires_on date,  
  risk_rating  text CHECK (risk_rating IN ('low', 'medium', 'high')),  
  jurisdiction char(2) NOT NULL,  
  relationship_manager_id bigint REFERENCES app_user(id),  
  onboarded_on date NOT NULL,  
  status       text NOT NULL DEFAULT 'active',  
  created_at   timestampz NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE product (  
  id          bigserial PRIMARY KEY,  
  code        text NOT NULL UNIQUE,  
  name        text NOT NULL,  
  class       text NOT NULL CHECK (class IN  
  
('current', 'savings', 'term_deposit', 'loan', 'overdraft')),  
  currency    char(3) NOT NULL,  
  gl_account_id bigint NOT NULL REFERENCES gl_account(id),  
  params      jsonb NOT NULL DEFAULT '{}' -- rates, tiers, fees,  
limits  
);
```

```
CREATE TABLE account (  
  id          bigserial PRIMARY KEY,  
  account_no  text NOT NULL UNIQUE,  
  customer_id bigint NOT NULL REFERENCES customer(id),  
  product_id  bigint NOT NULL REFERENCES product(id),  
  currency    char(3) NOT NULL,  
  branch_code text NOT NULL,  
  status      text NOT NULL DEFAULT 'active'  
              CHECK (status IN  
('active', 'dormant', 'restricted', 'frozen', 'closed')),  
  opened_on   date NOT NULL,
```

```

    closed_on      date
);

```

Product parameters live in jsonb rather than columns because every institution wants a different interest tier structure, and adding a column per variation is how a schema becomes unmaintainable. Parameters are validated against a JSON Schema per product class at write time.

5.3 Approvals, close and audit

```

CREATE TABLE approval_request (
    id          bigserial PRIMARY KEY,
    object_type text NOT NULL,           -- payment, journal,
customer...
    object_id   bigint NOT NULL,
    maker_id    bigint NOT NULL REFERENCES app_user(id),
    made_at     timestampz NOT NULL DEFAULT now(),
    checker_id  bigint REFERENCES app_user(id),
    decided_at  timestampz,
    status      text NOT NULL DEFAULT 'pending'
                CHECK (status IN
('pending', 'approved', 'rejected', 'expired')),
    reason      text,
    -- the four-eyes rule, in the schema rather than in a code comment
    CONSTRAINT checker_is_not_maker CHECK (checker_id IS NULL OR
checker_id <> maker_id)
);

```

```

CREATE TABLE accounting_period (
    id          bigserial PRIMARY KEY,
    starts_on   date NOT NULL,
    ends_on     date NOT NULL,
    status      text NOT NULL DEFAULT 'open'
                CHECK (status IN
('open', 'closing', 'closed', 'locked')),
    closed_by   bigint REFERENCES app_user(id),
    closed_at   timestampz
);

```

```

CREATE TABLE close_task (
    id          bigserial PRIMARY KEY,
    period_id   bigint NOT NULL REFERENCES accounting_period(id),
    title       text NOT NULL,
    owner_role  text NOT NULL,
    sequence    int NOT NULL,
    blocks_signoff boolean NOT NULL DEFAULT false,
    status      text NOT NULL DEFAULT 'pending',
    completed_by bigint REFERENCES app_user(id),
    completed_at timestampz
);

```

```

CREATE TABLE attachment (
  id          bigserial PRIMARY KEY,
  object_type text NOT NULL,
  object_id   bigint NOT NULL,
  filename    text NOT NULL,
  content_type text NOT NULL,
  byte_size   bigint NOT NULL,
  sha256       bytea NOT NULL,           -- proves the file is
unchanged
  storage_key  text NOT NULL,
  uploaded_by  bigint NOT NULL REFERENCES app_user(id),
  uploaded_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE audit_event (
  id          bigserial PRIMARY KEY,
  occurred_at timestampz NOT NULL DEFAULT now(),
  actor_id    bigint REFERENCES app_user(id),
  actor_label text NOT NULL,           -- survives user deletion
  action      text NOT NULL,          -- PAYMENT.APPROVE,
ROLE.GRANT...
  object_type text,
  object_id   bigint,
  source_ip   inet,
  user_agent  text,
  result      text NOT NULL CHECK (result IN
('ok', 'denied', 'error')),
  detail      jsonb,
  prev_hash   bytea NOT NULL,
  hash        bytea NOT NULL
);

```

The checker_is_not_maker constraint is the four-eyes rule expressed where it cannot be forgotten. A future developer who writes an approval endpoint without reading the policy document still cannot violate it — the database refuses.

5.4 Enforcing append-only

Application-level discipline is not enough; one careless migration undoes it. The journal and audit tables are protected at the database level:

```

REVOKE UPDATE, DELETE ON journal_entry, journal_leg, audit_event FROM
app_role;
GRANT INSERT, SELECT ON journal_entry, journal_leg, audit_event TO
app_role;

```

The application connects as app_role. Schema migrations use a separate, restricted account. A DELETE FROM journal_entry from application code fails as a permission error, not as a policy violation discovered at audit.

6. Security

Security requirements come from three places: the regulator, the customer's auditor, and the attacker. This section covers what the architecture must provide. **Every rule named here must be confirmed against current text with a compliance advisor licensed in the target jurisdiction before the corresponding module is built** — regulations change, and thresholds and deadlines in particular change often. Nothing in this document should be treated as legal advice or quoted to a regulator.

6.1 United States

Requirement	Source	What the platform must provide
Written information security programme; safeguards for customer information	GLBA Safeguards Rule (FTC)	Access controls, encryption, logging, incident response, vendor oversight
Authentication appropriate to risk for financial services access	FFIEC guidance on authentication and access	Multi-factor for all privileged access; layered controls; device and session management
Customer identification and anti-money-laundering programme	BSA / USA PATRIOT Act; FinCEN rules	Identity capture and verification, beneficial ownership, monitoring, suspicious activity reporting
Sanctions screening	OFAC	Screening of customers and payment counterparties against current lists; audited disposition of every hit
Electronic fund transfer error resolution and disclosure	Regulation E (EFTA)	Dispute case management with timestamps and evidence; customer notification records
Funds availability	Regulation CC	Hold logic on deposits, disclosed and applied consistently
Internal control over financial reporting (<i>public companies</i>)	SOX §404	Segregation of duties, immutable audit trail, evidenced close, access recertification

Requirement	Source	What the platform must provide
Card data handling (<i>only if cards are in scope</i>)	PCI DSS v4.x	Tokenised PANs, isolated cardholder data environment, quarterly scans
Service organisation controls, for selling to banks	SOC 2 (AICPA Trust Services Criteria)	Evidence of security, availability, confidentiality and processing integrity

Two practical notes. First, SOC 2 Type II is effectively a precondition for selling to a US financial institution, and it requires an *observation period* — typically several months of the controls actually operating. Start generating that evidence from the first production deployment, not when the first buyer asks. Second, if cards are not in phase 1 and 2, keep them out of scope deliberately: PCI DSS scope is the single largest avoidable compliance cost in this plan.

6.2 Global

Region	Requirement	Architectural consequence
EU / UK	PSD2 strong customer authentication; open banking APIs	MFA on customer payment initiation; consented third-party API access
EU / UK	GDPR	Lawful basis, data subject access and erasure, records of processing, breach notification. Erasure conflicts with ledger immutability — see 6.3
Global	ISO 20022	Payment message model; adopt the data model early even before a rail requires it
Global	SWIFT Customer Security Programme	Applies if SWIFT connectivity is built; annual attestation
Global	Basel III	Liquidity and capital reporting data must be derivable from the

Region	Requirement	Architectural consequence
Global	ISO/IEC 27001	ledger Information security management system; commonly requested in procurement
India	RBI directions; NPCI rules for UPI/IMPS/NEFT/RTGS	Rail integration is certification-gated, not just technical
India	Digital Personal Data Protection Act 2023	Consent, notice, breach reporting; similar tension with immutability

6.3 The conflict nobody mentions

An immutable ledger and a legal right to erasure are in direct tension. This must be designed for, not discovered.

The resolution is **crypto-shredding with a separated identity store**:

- The journal stores an internal customer key, never personal data
- Personal data lives in a separate store, encrypted per data subject
- An erasure request destroys that subject's key
- The financial record — amounts, dates, accounts, balances — survives intact and the ledger remains provable; the personal data is irrecoverable

Financial records are generally subject to statutory retention that overrides erasure for the retention period, and erasure is then applied to the personal data only. The exact interaction between retention and erasure obligations is jurisdiction-specific and is a question for counsel — but the architecture must be capable of either outcome from day one, because retrofitting it later means rewriting the ledger.

6.4 Controls the platform implements

Control	Implementation
Data at rest	AES-256; database and object storage encrypted; keys in a KMS or HSM
Data in transit	TLS 1.3 externally; mutual TLS between internal services
Key management	Rotation schedule, split custody, no key material in source or

Control	Implementation
Authentication	environment files Password (Argon2id) + TOTP; WebAuthn for privileged roles in phase 2
Session	Short idle timeout, absolute lifetime, device binding, concurrent session limits
Authorisation	Role-based, deny by default, evaluated server-side on every request
Segregation of duties	Enforced in the service layer and in the database constraint of 5.3
Privileged access	Break-glass accounts requiring dual approval; every session recorded
Audit	Append-only, hash-chained, includes denied attempts and read access to customer data
Input handling	Parameterised queries throughout; output encoding; strict CSP
Secrets	Vault or cloud secret manager; nothing in the repository; rotation on staff change
Backup	Encrypted, off-site, restore tested on a schedule — an untested backup is not a backup
DR	Documented RPO and RTO with a rehearsed failover, not an aspiration
Testing	Independent penetration test before each major release; findings tracked to closure

6.5 Threats specific to this domain

Generic web security is necessary but not sufficient. These are the ones that cost banks money:

- **Race conditions on balance.** Two concurrent debits both read a sufficient balance and both succeed. Mitigation: serialisable isolation or explicit row locking on the account, plus a load test that specifically attempts it.

- **Replay of a payment instruction.** The same signed request submitted twice. Mitigation: idempotency keys, mandatory on every state-changing API call.
- **Rounding and truncation drift.** Interest and FX calculations that lose a paisa per transaction. Mitigation: banker's rounding, defined precision at every boundary, a daily reconciliation job that would surface the drift.
- **Privilege creep.** Staff accumulate rights across role changes until someone can both make and check. Mitigation: periodic access recertification, and an alert when one user's rights span a segregation boundary.
- **Insider access to customer data.** Mitigation: read access to customer records is itself an audited event, and bulk export requires approval.

7. API design

REST over HTTPS, JSON, versioned in the path. OAuth 2.1 client credentials for service consumers; session authentication for the back office.

POST	/api/v1/customers	create customer (maker)
GET	/api/v1/customers/{cif}	full customer profile
POST	/api/v1/customers/{cif}/kyc	submit KYC evidence
GET	/api/v1/accounts?cif=...	accounts for a customer
POST	/api/v1/accounts	open account (maker)
GET	/api/v1/accounts/{no}/statement	paginated statement
POST	/api/v1/payments	create payment (maker)
POST	/api/v1/payments/{id}/approve	release payment (checker)
POST	/api/v1/payments/{id}/reject	reject with reason
GET	/api/v1/journal?from=&to=&account=	query the ledger
POST	/api/v1/journal	manual entry (maker, restricted)
POST	/api/v1/journal/{ref}/reverse	post a reversal
GET	/api/v1/reports/trial-balance	as at a date
GET	/api/v1/periods/{id}/close-tasks	close checklist state
POST	/api/v1/periods/{id}/sign-off	sign off the period (checker)

Four rules that apply to every endpoint:

Idempotency is mandatory on writes. Every POST accepts an Idempotency-Key header. A repeat of the same key returns the original response and does not act twice. A payments API without this will eventually double-pay someone.

Money is a string. Amounts serialise as {"amount": "12500.0000", "currency": "INR"}. Never a JSON number — JSON numbers are IEEE 754 doubles in most parsers, and that reintroduces the floating-point problem at the API boundary after you carefully avoided it in the database.

Errors are specific and stable. 422 with a machine-readable code (INSUFFICIENT_BALANCE, PERIOD_CLOSED, SELF_APPROVAL_BLOCKED) and a human message. Integrators code against the code, so it never changes meaning.

Every write is audited before it responds. The audit record is written in the same transaction as the change. If the audit write fails, the change fails.

7.1 Events

Emitted for consumers, once phase 2 introduces them. Events are notifications, not the source of truth — a consumer that missed one re-reads the API.

customer.created	account.opened	
account.status_changed		
payment.created	payment.approved	payment.settled
payment.rejected	journal.posted	journal.reversed
kyc.expiring	period.closed	exception.raised

8. Roles and permissions

Deny by default. A role grants only what its job requires, and no role can both make and check the same object class.

Role	Can do	Cannot do
Teller	Cash in/out, enquiry, cheque deposit	Approve own transaction; open accounts
Branch Manager	Teller rights, overrides, account-opening approval	Approve an override they raised
Payments Maker	Create and amend payments before release	Release any payment
Payments Checker	Approve or reject payments	Create a payment they will approve
Operations Manager	Exception queue, workflow, read-all	Post to the ledger
Controller	Journal entries, close tasks, sign-off	Bypass four-eyes on postings

Role	Can do	Cannot do
Compliance / FCC	AML disposition, sanctions review, filings	Operational or financial transactions
System Admin	Roles, configuration, technical operations	Any financial transaction, any approval
Auditor	Read everything, including the audit trail	Change anything

The System Admin restriction is deliberate and is the one most often compromised in practice. The person who can grant permissions must not be able to move money — otherwise every other control is decorative.

9. Interface design

9.1 Design rationale

The prototype's visual direction is a considered position, and it is documented here so it can be argued with rather than guessed at.

Back-office banking software is used for seven hours a day by people who are measured on throughput and accuracy. That leads to specific choices:

Decision	Reason
Warm paper background, not grey-blue	Reduces glare over a long shift; distinguishes the product from the identical blue-grey of every SaaS dashboard
Hairline rules, not drop shadows	Shadows add visual noise at high information density; rules organise without competing
Tabular figures everywhere	Digits align vertically, so a column of amounts can be scanned rather than read
Monospace for amounts, IDs and hashes	Transposition errors become visible; a wrong digit count is obvious
Serif headings against a sans interface	Institutional weight without the dated look of a serif body; also makes the product visually memorable in a deck
Graphite navigation rail	Anchors the eye; separates chrome from data so the data is what is

Decision	Reason
Brass accent, used sparingly	bright Reads as financial rather than technological; kept to focus states and one highlight so it retains meaning
Dense tables, generous panel spacing	Density where data lives, air where the eye rests

9.2 Colour and accessibility

The data palette is pine #127A5B, brass #AD7A10 and blue #2C6FB0. It was not chosen by eye — it was run through a colour-vision validator and passes on lightness banding, chroma, colour-blind separation between every adjacent pair, normal-vision separation, and contrast against the page surface.

Status is never communicated by colour alone: every state carries a text label, and the chart series carry both a legend and direct end labels.

9.3 Screen inventory

Phase 1 screens, all present in the prototype:

1. Sign-in — credential step
2. Sign-in — TOTP second factor with device trust
3. Operations dashboard — KPI strip, trend chart, close progress, funding mix, exception queue
4. Customer 360 — searchable register with a full profile, exposure, linked accounts and document vault
5. Accounts and deposits — register plus statement view
6. Payments — queue with maker-checker enforcement visibly demonstrated
7. General ledger — journal with the hash chain rendered and verifiable
8. Financial close — trial balance, task checklist, evidence upload, sign-off
9. Audit and security — audit trail, control state, role matrix

Phase 2 adds: loan origination and servicing, term deposit management, branch teller and cash position, reconciliation workbench, customer web and mobile.

10. The working prototype

Built and delivered alongside this document. Not a mockup — a running application.

What works, for real:

- Two-step authentication. The TOTP step accepts only the correct code, rejects anything else, auto-advances between digit boxes and handles paste.
- Every screen listed in 9.3, navigable, with a working router and deep links.
- The trend chart is interactive: hovering gives a crosshair and the exact figure for each series in that month.
- The ledger's hash chain verifies on demand, walking each entry and reporting the result.
- Payment approve and reject act on the queue. The payment created by the signed-in operator cannot be approved — the button is replaced by the reason.
- Documents drag-and-drop onto a close task, with size and type shown, and can be removed.
- The close checklist, trial balance and evidence counts are consistent with each other.

Data integrity in the demonstration set — checked, not assumed:

- The trial balance balances exactly: debits ₹5,64,85,19,00,000, credits the same, difference zero
- Deposits and advances on the dashboard reconcile to the deposit and loan lines of that trial balance
- The CASA ratio shown is computed from those lines, not typed in
- Every journal entry's legs balance to zero
- Each entry's stored previous-hash equals the hash of the entry before it — the chain is continuous

All data is synthetic. There are no real customers, accounts or transactions in it, and no real personal data of any kind.

What is deliberately not there: no database, no server-side logic, no real authentication. It is a front end over a fixed dataset, which is the correct scope for establishing product and visual direction before backend work starts.

11. Delivery phases

Each phase ends in something demonstrable. No phase is a prerequisite that produces nothing visible.

Phase 0 — Direction (*delivered*)

This plan and the working prototype.

Phase 1 — The core, end to end

The vertical slice: a real institution could keep its books on this.

- PostgreSQL schema of section 5, with append-only grants
- Posting engine with balance enforcement and hash chaining
- Authentication, TOTP, sessions, roles, maker-checker gate
- Customer 360 with KYC states and document vault
- Chart of accounts, deposit products, account opening
- Transaction posting, statements, running balances
- Operations dashboard against live data
- Financial close: period control, trial balance, checklist, evidence, sign-off
- Reporting: trial balance, P&L, balance sheet, statements — CSV and PDF
- Audit trail, hash-chained, with a verification job
- Test suite covering the posting engine, four-eyes and the close

Exit criterion: a full month is opened, transacted on, and closed with sign-off, and the closing trial balance balances without manual intervention.

Phase 2 — Operations

- Term deposits: maturity, renewal, premature closure
- Lending: origination, schedules, disbursement, repayment, delinquency
- Payments hub: rail-agnostic core with adapters; ISO 20022 message model
- Reconciliation workbench: nostro, rails, card networks
- Branch and teller: cash drawer, vault, denominations, day open/close
- Customer-facing web and mobile: balances, statements, payments
- Public REST API with idempotency and OAuth 2.1
- WebAuthn for privileged roles

Exit criterion: a payment is originated by a customer, approved under four-eyes, settled, reconciled and reflected in the close — without a manual step anywhere in that chain.

Phase 3 — Depth, driven by the first real customer

AML and sanctions; regulatory reporting for one named jurisdiction; treasury; enterprise ERP over the same ledger; BI on a read replica; cards, if a buyer funds the PCI DSS scope; migration tooling; the AI features of 3.4.

Phase 3 is deliberately not specified in detail here. Specifying it now would be guessing. It should be written against the first paying customer's actual

requirements, because that is the only reliable source of truth about which of these modules matters.

11.1 Sequencing rules

1. **The ledger is first and is never retrofitted.** Every other module posts to it. Building any module before it means rewriting that module.
2. **Security is built in phase 1, not bolted on in phase 3.** Adding MFA, audit and segregation of duties to a working system costs several times what building with them costs.
3. **One rail before many.** The second payment rail takes a fraction of the first if the first was built rail-agnostic, and three times as long if it was not.
4. **Never build regulatory reporting speculatively.** Build it against a live requirement from a real institution in a named jurisdiction. Reports built from a specification nobody has to file are always wrong.

12. Testing and acceptance

12.1 Layers

Layer	Covers	Standard
Unit	Posting engine, interest and schedule maths, rounding, FX	100% on money-touching code. Not a target — a gate
Integration	Postings, approvals, close, statements against a real database	Every workflow, both success and refusal
Contract	API request and response shapes	Every public endpoint
Security	Authentication, authorisation, four-eyes, injection, session handling	Every role boundary tested from the wrong side
Concurrency	Simultaneous debits, double approval, replayed idempotency keys	Explicit adversarial tests, not load tests
Reconciliation	Cached balances against journal aggregates	Automated, on every build and nightly
User acceptance	Business scenarios run by the customer's own	Signed off per scenario

Layer	Covers	Standard
	staff	

12.2 Tests that must exist before phase 1 is accepted

These are the ones that catch the failures that matter. Each must be written to *fail* if the control is removed:

- An unbalanced journal entry cannot be persisted
- A journal entry cannot be updated or deleted, including by direct SQL as the application user
- Modifying a stored entry breaks chain verification and the verifier says so
- A maker cannot approve their own payment — through the UI *and* by calling the API directly with the maker's own token
- A posting to a closed period is refused
- Two concurrent debits totalling more than the balance: exactly one succeeds
- A replayed idempotency key returns the first response and does not post twice
- Interest accrual over a full year reconciles to the expected total to the paisa
- A user whose role is revoked mid-session loses access at the next request
- Cached balances equal journal aggregates after a randomised transaction run

A control that has never been observed failing has not been tested. Each of these should be run once with the control disabled, to prove the test can fail.

12.3 Acceptance criteria for phase 1

Signed off when all of the following hold:

1. A month is opened, transacted on and closed, with a balancing trial balance
2. Every test in 12.2 passes, and each has been demonstrated to fail with its control removed
3. Chain verification passes over the full journal
4. A restore from backup into a clean environment reproduces the ledger exactly, verified by comparing the final hash
5. An independent security review has been completed and its findings closed or formally accepted
6. The customer's own staff have run their scenarios and signed them off

13. Team

13.1 Minimum viable team for phase 1

Role	Allocation	Responsible for
Backend engineer (lead)	Full time	Posting engine, schema, services
Backend engineer	Full time	Modules, API, integrations
Frontend engineer	Full time	Back-office interface
QA engineer	Full time from mid-phase	The suite in section 12
Banking domain advisor	Part time	Correctness of accounting and process
DevOps / security	Part time	Deployment, secrets, backup, hardening

The domain advisor is the role most often cut and the most expensive to omit. Someone who has personally closed a bank's books will find, in an afternoon, errors that six months of user testing would not surface. It does not need to be full time. It does need to be someone who has actually done the job.

13.2 Growth

Phase 2 adds a second frontend engineer, a mobile engineer and a full-time DevOps engineer. Phase 3 adds a compliance specialist and a data engineer. Resist growing faster than that: on a system where correctness is the product, adding engineers to a moving domain model reliably slows delivery down.

14. Risks

Risk	Likelihood	Impact	Response
Scope expands toward the full 29-module list	High	Fatal	Phase gates; nothing enters a phase mid-phase
Ledger design changes after other modules depend on it	Medium	Severe	Get section 5 reviewed by a domain advisor before any code
Regulatory requirement discovered late	High	Severe	Compliance review at each phase boundary,

Risk	Likelihood	Impact	Response
First customer wants a module from phase 3	High	Moderate	not at the end Price and schedule it as an addition; do not silently absorb it
Floating-point or rounding drift found in production	Low	Severe	numeric throughout, plus the reconciliation job of 12.2
Security finding after go-live	Medium	Severe	Independent test before each release; findings tracked to closure
Key person leaves	Medium	Moderate	This document; code review on everything; no single-owner modules
Building for a buyer who does not exist	Medium	Fatal	Find the design partner during phase 1, not after phase 3

The last one is the real risk. A technically excellent platform with no design partner is the most common way this kind of project fails. The highest-value activity running in parallel with phase 1 is finding one institution willing to be the first customer and to tell you the truth about what they need.

15. Immediate next steps

1. **Send the UI video.** The interface direction in section 9 is my own reasoning. If the video shows something different, redirecting it now costs an afternoon; redirecting it after phase 1 costs weeks.
2. **Review the prototype and mark it up.** Specifics are more useful than general approval — the wrong field on the customer screen, a missing column, a term your market uses differently.
3. **Confirm the phase 1 scope** in section 11 as written, or state what moves.
4. **Confirm the stack** in section 4.1, or state a preference.
5. **Decide the first target segment** from section 2.3. This changes what phase 2 contains.

6. **Find the domain advisor.** Someone who has closed a bank's books. Needed before the schema is frozen.

The investor deck and the user deck come after phase 1, once there is a working system to show. A deck built on a prototype is a pitch; a deck built on a system that closed a real month is evidence, and the difference is the whole point.

Prepared by Anirudha Talmale · 30 August 2026 · Version 1.0

Regulatory references in section 6 identify the controls that shape this architecture. They are not legal advice, and must be confirmed against current text with a compliance advisor licensed in the target jurisdiction before the corresponding module is built.